In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

AGENTS.md

Found in folder: **/**

This file provides project context and guidelines for developers working on the Todo app. It outlines the technology stack, available commands, and coding conventions. It also includes a placeholder section for critical rules to be added later.

AGENTS.md

This file gives Codex context about the project.

Project

A beginner-friendly Todo app built with React and Vite, used as a starter

project **for** a Codex course. Students extend the app by adding missing

features — guided by Codex.

Stack

| Layer **|** Choice **|**

|---|---|

| UI **|** React 19 **|**

| Bundler **|** Vite 7 **|**

| Styling **|** CSS Modules **(**plain CSS only**) |**

| Tests **|** Vitest + Testing Library **|**

| Persistence **|** localStorage **|**

No backend. No routing. No auth. No external UI libraries.

Commands

| Command **|** Purpose **|**

|---|---|

| `npm run dev` **|** Start dev server **(**localhost:5173**)|**

| `npm run build` | Production build |

| `npm test` | Run tests **in** watch mode |

| `npm run test:ui` | Open the Vitest browser UI |

Conventions

- All state lives **in** `App.jsx`; components receive data and callbacks as props

- One component per file; its CSS module lives **in** the same directory

- Task shape: `**{** id: string, text: string, completed: boolean **}**`

- See `/docs/ui.md` **for** styling rules

- See `/docs/data-fetching.md` **for** state and persistence rules

Critical Rules

<!-- Students: fill **this** section **in** during the Codex course --**>**

README.md

Found in folder: **/**

This file is a project README that documents a Todo app starter kit. It lists existing features, missing functionality to be added as exercises, and provides setup instructions. The document also outlines the project structure and technology stack used.

Todo App — Claude Code Course Starter

A deliberately incomplete Todo app. You will use **Claude Code** to add the

missing features listed below.

Getting started

```bash

npm install

npm run dev       # http://localhost:5173

```

Running tests

```bash

npm test          # watch mode
```

npm run test:ui   # browser UI

```

One test passes. One test fails — that is intentional. Fixing it is your

first exercise.

## What already works

| Feature | File |

|---|---|

| Add a task | `TaskInput.jsx` |

| Mark complete (checkbox) | `TaskItem.jsx` |

| Delete a task | `TaskItem.jsx` |

| Character counter (max 100, turns red) | `TaskInput.jsx` |

| Filter bar — All / Active / Completed | `FilterBar.jsx` |

| localStorage persistence | `App.jsx` |

## What is missing (your exercises)

- **Empty-input validation** — clicking Add Task with a blank field still
  creates an empty task. Add a guard in `TaskInput.jsx` to prevent this.
  The failing test in `src/__tests__/tasks.test.jsx` will go green once you do.
- **Task editing** — double-clicking a task text should let you edit it
  in place and save on Enter / blur.
- **Due dates** — tasks have no deadline field. Add a date picker to
  `TaskInput` and display the date in `TaskItem`.
- **Drag to reorder** — tasks are fixed in insertion order.

## Project structure

```

src/

  components/

TaskInput.jsx + TaskInput.module.css

TaskList.jsx + TaskList.module.css

TaskItem.jsx + TaskItem.module.css

FilterBar.jsx + FilterBar.module.css

App.jsx + App.module.css

main.jsx

index.css

docs/

ui.md — styling conventions

data-fetching.md — state and persistence conventions

CLAUDE.md — context file **for** Claude Code

vitest.config.js

```

## Stack

- React 19 + Vite 7

- CSS Modules only **(**no Tailwind, no UI libraries**)**

- Vitest + Testing Library

- localStorage **(**no backend**)**

**data-fetching.md**

Found in folder: **/docs**

This markdown file documents the state management and data flow conventions for a todo app, describing the task object structure, persistence using localStorage, and how tasks are filtered and mutated. It explains that all state is centralized in App.jsx and outlines the callback functions for task operations. The document also notes current limitations and features not yet implemented.

# Data & State Conventions

## State Shape
```

All app state lives in `App.jsx`. There is no global state manager.

```js
// Task object — the single source of truth
{
  id: string,       // crypto.randomUUID(), set at creation time
  text: string,     // 1–100 characters (MAX_LENGTH enforced in TaskInput)
  completed: boolean // toggled by the checkbox
}
```

The active filter (`'all' | 'active' | 'completed'`) is also stored in `App`
but is never persisted — it resets to `'all'` on page load.

Persistence

Tasks are persisted to `localStorage` under the key `"todo-tasks"`.

Read — lazy initialiser so it runs exactly once on mount:

```js
const [tasks, setTasks] = useState(() => {
  const stored = localStorage.getItem('todo-tasks')
  return stored ? JSON.parse(stored) : []
})
```

Write — effect that syncs whenever `tasks` changes:

```js
useEffect(() => {
  localStorage.setItem('todo-tasks', JSON.stringify(tasks))
}, [tasks])
```

Task Operations

All mutations live **in** `App.jsx` and are passed to children as callbacks:

Prop	Signature	Behaviour
`onAdd`	`(text: string) => void`	Prepends a **new** task to the array
`onToggle`	`(id: string) => void`	Flips `completed` on one task
`onDelete`	`(id: string) => void`	Removes the task from the array

Mutations always use the functional updater form (`setTasks(prev => ...)`) to avoid stale-closure bugs.

Filtering

Filtering is derived — it does **not** mutate `tasks`:

```js
const filteredTasks = tasks.filter(task => {
  if (filter === 'active') return !task.completed
  if (filter === 'completed') return task.completed
  return true
})
```

Only `filteredTasks` is passed to `<TaskList>`. The full `tasks` array is used **for** the count **in** `<FilterBar>`.

What Is Not Here (yet)

- No API calls — everything is local; no `fetch`, no async state
- No loading or error states
- No optimistic updates
- No task editing, due dates, or ordering — those are student exercises **in** the Codex course

File extension: md

ui.md

Found in folder: **/docs**

This file documents the user interface design system and component structure for the project. It defines visual conventions like colors, spacing, and layout rules. It also outlines the component architecture and provides instructions for adding new components.

UI Conventions

Layout

- All components live **in** `src/components/`

- Each component has a matching `.module.css` file **in** the same directory

- Max content width: `600px`, centered via `margin: auto` **in** `App.module.css`

- Page background: `#f3f4f6` (set in `src/index.css`)

Colour Palette

Token	Value	Used **for**
Primary	`#6366f1`	Buttons, focus rings, checkboxes
Primary dark	`#4f46e5`	Button hover
Danger	`#ef4444`	Delete hover, character limit
Danger light	`#fee2e2`	Delete button hover background
Border	`#d1d5db`	Default input / pill borders
Border inner	`#e5e7eb`	Task item border
Text muted	`#9ca3af`	Empty state, count, delete icon
Text base	`#1a1a1a`	Body copy
Surface	`#fff`	Cards, inputs

Border Radii

Element	Radius
Inputs, buttons	`6px`

| Task list items | `8px` |

| Filter pills | `20px` |

| Delete button | `4px` |

Interaction

- Every interactive element has a `transition` on whichever property changes

- Default transition duration: `0.15s`

- Checkboxes use `accent-color: #6366f1`

Component Hierarchy

```
App
 ├── TaskInput    — input field, Add Task button, character counter
 ├── FilterBar    — All / Active / Completed pills + task count
 └── TaskList
   └── TaskItem  — checkbox, task text, delete button
```

Adding a New Component

1. Create `src/components/ComponentName.jsx`

2. Create `src/components/ComponentName.module.css` alongside it

3. Import CSS with `import styles from './ComponentName.module.css'`

4. Apply **class** names with `className={styles.className}`

5. Wire it into the appropriate parent component; lift any **new** state to `App.jsx`

App.jsx

Found in folder: **/src**

This code manages a todo list app by storing tasks in local storage and allowing users to add, toggle, delete, and edit tasks. It filters tasks based on their completion status and displays them through child components. The app maintains state for tasks and a filter, with effects to persist tasks to local storage.

```
import { useState, useEffect } from 'react'
import TaskInput from './components/TaskInput'
import TaskList from './components/TaskList'
import FilterBar from './components/FilterBar'
import styles from './App.module.css'
function App() {
 // Initialize tasks from localStorage or empty array
 const [tasks, setTasks] = useState(() => {
  const stored = localStorage.getItem('todo-tasks')
  return stored ? JSON.parse(stored) : []
 })
 const [filter, setFilter] = useState('all')
 // Persist tasks to localStorage whenever they change
 useEffect(() => {
  localStorage.setItem('todo-tasks', JSON.stringify(tasks))
 }, [tasks])
 const addTask = (text) => {
  setTasks(prev => [
   { id: crypto.randomUUID(), text, completed: false },
   ...prev,
  ])
 }
 const toggleTask = (id) => {
  setTasks(prev =>
   prev.map(task =>
    task.id === id ? { ...task, completed: !task.completed } : task
```

```
    )
  )
}
const deleteTask = (id) => {
  setTasks(prev => prev.filter(task => task.id !== id))
}
const editTask = (id, text) => {
  setTasks(prev =>
    prev.map(task =>
      task.id === id ? { ...task, text } : task
    )
  )
}
// Filter tasks based on current filter selection
const filteredTasks = tasks.filter(task => {
  if (filter === 'active') return !task.completed
  if (filter === 'completed') return task.completed
  return true
})
return (
  <div className={styles.app}>
    <h1 className={styles.title}>Todo</h1>
    <TaskInput onAdd={addTask} />
    <FilterBar filter={filter} onFilterChange={setFilter} taskCount={tasks.length} />
    <TaskList
      tasks={filteredTasks}
```

```
      onToggle={toggleTask}
      onDelete={deleteTask}
      onEdit={editTask}
    />
  </div>
 )
}
export default App
/*
Optional due-date reference (inactive):
const addTask = (text, dueDate) => {
 setTasks(prev => [
  { id: crypto.randomUUID(), text, dueDate, completed: false },
  ...prev,
 ])
}
*/
```

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component into the DOM. It uses React's StrictMode to highlight potential problems during development. The App component is wrapped inside a root DOM element with the id 'root'.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
// Create a React root at the DOM element with id 'root' and render the app
```

```
createRoot(document.getElementById('root')).render(
 <StrictMode>
  <App />
 </StrictMode>
)
```

FilterBar.jsx

Found in folder: **/src/components**

This code renders a set of filter buttons and a task counter. It maps over a list of filter types to create interactive buttons, highlighting the currently active one. It also displays the total number of tasks, using the correct singular or plural form.

```
import styles from './FilterBar.module.css'

// Predefined list of filter options
const FILTERS = ['all', 'active', 'completed']

export default function FilterBar({ filter, onFilterChange, taskCount }) {
 return (
  <div className={styles.bar}>
   {FILTERS.map(f => (
    <button
     key={f}
     // Apply 'active' style if this button matches the current filter
     className={`${styles.filterButton} ${filter === f ? styles.active : ''}` }
     onClick={() => onFilterChange(f)}
    >
     {/* Capitalize the first letter of the filter name for display */}
     {f.charAt(0).toUpperCase() + f.slice(1)}
    </button>
   ))}
```

```
    <span className={styles.count}>
     {/* Display count with correct pluralization */}
     {taskCount} {taskCount === 1 ? 'task' : 'tasks'}
    </span>
   </div>
 )
}
```

TaskInput.jsx

Found in folder: **/src/components**

This code provides a text input and button for adding new tasks, with validation and a character counter. It trims whitespace from the input before adding and prevents empty submissions. The component also includes visual feedback when approaching the character limit.

```
import { useState } from 'react'
import styles from './TaskInput.module.css'
const MAX_LENGTH = 100 // Maximum allowed characters for a task
export default function TaskInput({ onAdd }) {
 const [text, setText] = useState('')
 const handleAdd = () => {
  const nextText = text.trim()
  if (!nextText) return // Prevent adding empty tasks
  onAdd(nextText)
  setText('') // Clear input after adding
 }
 return (
  <div className={styles.wrapper}>
   <div className={styles.inputRow}>
```

```
      <input
        className={styles.input}
        type="text"
        value={text}
        onChange={e => setText(e.target.value)}
        onKeyDown={e => e.key === 'Enter' && handleAdd()} // Allow adding with Enter key
        placeholder="What needs to be done?"
        maxLength={MAX_LENGTH}
        aria-label="New task"
      />
      <button className={styles.button} onClick={handleAdd}>
        Add Task
      </button>
    </div>
    {/* Character counter with visual warning when approaching limit */}
    <p className={`${styles.counter} ${text.length >= MAX_LENGTH ? styles.atLimit : ''}`}>
      {text.length} / {MAX_LENGTH}
    </p>
  </div>
 )
}
/*
Optional due-date reference (inactive):
const [dueDate, setDueDate] = useState('')
const handleAdd = () => {
  const trimmedText = text.trim()
```

```
  if (!trimmedText) return
  onAdd(trimmedText, dueDate || null)
  setText('')
  setDueDate('')
}
<input
  className={styles.dateInput}
  type="date"
  value={dueDate}
  onChange={e => setDueDate(e.target.value)}
  aria-label="Due date"
/>
*/
```

TaskItem.jsx

Found in folder: **/src/components**

This component renders a single task item with a checkbox, editable text, and a delete button. It allows toggling completion, editing via double-click, and saving/canceling edits with Enter/Escape keys or blur events. The editing state manages focus and prevents unwanted save on certain interactions.

```
import { useEffect, useRef, useState } from 'react'
import styles from './TaskItem.module.css'
export default function TaskItem({ task, onToggle, onDelete, onEdit }) {
  const [isEditing, setIsEditing] = useState(false)
  const [draft, setDraft] = useState(task.text) // Local copy for editing
  const inputRef = useRef(null)
  const ignoreBlurRef = useRef(false) // Prevents save on blur for cancel actions
  // Focus and select text when entering edit mode
```

```
useEffect(() => {
  if (isEditing) {
    inputRef.current?.focus()
    inputRef.current?.select()
  }
}, [isEditing])
const startEditing = () => {
  ignoreBlurRef.current = false
  setDraft(task.text)
  setIsEditing(true)
}
const saveEdit = () => {
  const nextText = draft.trim()
  if (nextText) {
    onEdit(task.id, nextText)
  }
  setIsEditing(false)
}
const cancelEdit = () => {
  ignoreBlurRef.current = true // Signal to ignore the upcoming blur event
  setDraft(task.text)
  setIsEditing(false)
}
const handleKeyDown = (e) => {
  if (e.key === 'Enter') {
    e.preventDefault()
```

```
    saveEdit()
  }
  if (e.key === 'Escape') {
    e.preventDefault()
    cancelEdit()
  }
}
const handleBlur = () => {
  if (ignoreBlurRef.current) {
    ignoreBlurRef.current = false
    return
  }
  saveEdit() // Save on blur unless canceled
}
return (
  <li className={`${styles.item} ${task.completed ? styles.completed : ''}`}>
    <input
      type="checkbox"
      className={styles.checkbox}
      checked={task.completed}
      onChange={() => onToggle(task.id)}
      aria-label={`Mark "${task.text}" as ${task.completed ? 'active' : 'complete'}`}
    />
    {isEditing ? (
      <input
        ref={inputRef}
```

```
        className={styles.editInput}
        type="text"
        value={draft}
        onChange={e => setDraft(e.target.value)}
        onKeyDown={handleKeyDown}
        onBlur={handleBlur}
        aria-label={`Edit "${task.text}"`}
      />
    ) : (
      <span className={styles.text} onDoubleClick={startEditing}>
        {task.text}
      </span>
    )}
    <button
      className={styles.deleteButton}
      onClick={() => onDelete(task.id)}
      aria-label={`Delete "${task.text}"`}
    >
      ×
    </button>
  </li>
 )
}
/*
Optional due-date reference (inactive):
const dueDateFormatter = new Intl.DateTimeFormat('en-GB', {
```

```
  day: 'numeric',
  month: 'short',
  year: 'numeric',
  timeZone: 'UTC',
})
function formatDueDate(dueDate) {
  return dueDateFormatter.format(new Date(`${dueDate}T00:00:00Z`))
}
<span className={styles.details}>
  <span className={styles.text}>{task.text}</span>
  {task.dueDate && (
    <time className={styles.dueDate} dateTime={task.dueDate}>
      Due: {formatDueDate(task.dueDate)}
    </time>
  )}
</span>
*/
```

TaskList.jsx

Found in folder: **/src/components**

This component renders a list of tasks. If the list is empty, it displays a message. Otherwise, it maps each task to a TaskItem component, passing down the task data and callback functions.

```
import TaskItem from './TaskItem'
import styles from './TaskList.module.css'
export default function TaskList({ tasks, onToggle, onDelete, onEdit }) {
  // Show a message if there are no tasks
  if (tasks.length === 0) {
```

```
  return <p className={styles.empty}>No tasks here.</p>
 }
 // Render a list of TaskItem components for each task
 return (
  <ul className={styles.list}>
   {tasks.map(task => (
    <TaskItem
     key={task.id} // Unique key for React's list rendering
     task={task}
     onToggle={onToggle}
     onDelete={onDelete}
     onEdit={onEdit}
    />
   ))}
  </ul>
 )
}
```

TaskNote.jsx

Found in folder: **/src/components**

This code defines a React component that renders a paragraph element to display a task's note text. It applies CSS module styles to both the paragraph and the note text span. The component receives a `task` object as a prop and extracts the `note` property from it.

```
import styles from './TaskNote.module.css'

export default function TaskNote({ task }) {
 // Renders a paragraph containing the note text from the provided task object
 return (
  <p className={styles.note}>
```

```
    <span className={styles.text}>{task.note}</span>
   </p>
  )
}
```

tasks.test.jsx

Found in folder: **/src/__tests__**

This code tests a Todo app's UI behavior, including rendering, adding tasks, and editing tasks. It ensures tasks are not added when the input is empty and that edits are saved or canceled correctly. The tests also clear localStorage before each test to maintain isolation.

```
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, beforeEach } from 'vitest'
import App from '../App'
// Clear any stored data before each test to ensure a clean state
beforeEach(() => {
 localStorage.clear()
})
describe('Todo App', () => {
 // PASSING: the app should render with a visible heading.
 it('renders the app title', () => {
  render(<App />)
  expect(screen.getByRole('heading', { name: 'Todo' })).toBeInTheDocument()
 })
 // FAILING: this test documents a missing feature.
 //
 // The app currently adds a task even when the input is empty.
 // Your job: open TaskInput.jsx and add a guard in handleAdd() so that
 // clicking "Add Task" with a blank field does nothing.
```

```
// Once you do, this test will go green.
it('does not add a task when the input is empty', () => {
  render(<App />)
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
  const items = screen.queryAllByRole('listitem')
  expect(items).toHaveLength(0)
})
it('saves an edited task when Enter is pressed', () => {
  render(<App />)
  fireEvent.change(screen.getByLabelText('New task'), {
    target: { value: 'Buy milk' },
  })
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
  fireEvent.doubleClick(screen.getByText('Buy milk'))
  const editInput = screen.getByLabelText('Edit "Buy milk"')
  fireEvent.change(editInput, { target: { value: 'Buy oat milk' } })
  fireEvent.keyDown(editInput, { key: 'Enter' })
  expect(screen.getByText('Buy oat milk')).toBeInTheDocument()
  expect(screen.queryByText('Buy milk')).not.toBeInTheDocument()
})
it('saves an edited task when the input loses focus', () => {
  render(<App />)
  fireEvent.change(screen.getByLabelText('New task'), {
    target: { value: 'Read docs' },
  })
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
```

```
  fireEvent.doubleClick(screen.getByText('Read docs'))
  const editInput = screen.getByLabelText('Edit "Read docs"')
  fireEvent.change(editInput, { target: { value: 'Read React docs' } })
  fireEvent.blur(editInput)
  expect(screen.getByText('Read React docs')).toBeInTheDocument()
  expect(screen.queryByText('Read docs')).not.toBeInTheDocument()
 })
 it('cancels an edited task when Escape is pressed', () => {
  render(<App />)
  fireEvent.change(screen.getByLabelText('New task'), {
   target: { value: 'Walk home' },
  })
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
  fireEvent.doubleClick(screen.getByText('Walk home'))
  const editInput = screen.getByLabelText('Edit "Walk home"')
  fireEvent.change(editInput, { target: { value: 'Drive home' } })
  fireEvent.keyDown(editInput, { key: 'Escape' })
  expect(screen.getByText('Walk home')).toBeInTheDocument()
  expect(screen.queryByText('Drive home')).not.toBeInTheDocument()
 })
})
/*
Optional due-date reference (inactive):
it('adds and displays an optional due date', () => {
 render(<App />)
 fireEvent.change(screen.getByLabelText('New task'), {
```

```
  target: { value: 'Submit project' },
 })
 fireEvent.change(screen.getByLabelText('Due date'), {
  target: { value: '2026-07-24' },
 })
 fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
 expect(screen.getByText('Submit project')).toBeInTheDocument()
 expect(screen.getByText('Due: 24 Jul 2026')).toBeInTheDocument()
})
*/
```

index.html

Found in folder: **/**

This HTML file defines the basic structure of a web page, setting up metadata, linking resources, and providing a root element for a React application to render into. It specifies the document type, language, and viewport settings for responsive design. The page loads a JavaScript module as the main entry point for the application.

```
<!doctype html>
<html lang="fr">
 <head>
  <meta charset="UTF-8" />
  <link rel="icon" type="image/svg+xml" href="/vite.svg" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Vite + React</title>
 </head>
 <body>
  <!-- Root element where the React application will be mounted -->
  <div id="root"></div>
```

```
    <!-- Entry point script for the application, using ES modules -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

App.module.css

Found in folder: **/src**

This CSS code defines styles for two classes: `.app` centers a container with a maximum width and sets the font, while `.title` styles a large, bold, centered heading.

```
.app {
  max-width: 600px;        /* Limits container width */
  margin: 48px auto;       /* Centers container vertically and horizontally */
  padding:  0 16px;        /* Adds horizontal padding */
  font-family: system-ui, -apple-system, sans-serif; /* Uses system fonts */
}
.title {
  font-size: 2rem;         /* Large font size */
  font-weight: 700;        /* Bold text */
  text-align: center;      /* Centers text */
  margin: 0 0 32px;        /* Adds bottom margin only */
  color: #1a1a1a;          /* Dark gray color */
  letter-spacing: -0.5px;   /* Slightly reduces letter spacing */
}
```

index.css

Found in folder: **/src**

This code sets global styling rules for all elements and the body. It applies border-box sizing to everything, removing default margin from the body and setting a light gray background with dark text. It also enables font smoothing for better text rendering on WebKit browsers.

```
*,
*::before,
*::after {
  box-sizing: border-box; /* Includes padding and border in element's total width/height */
}
body {
  margin: 0; /* Removes default browser margin */
  background: #f3f4f6; /* Light gray background */
  color: #1a1a1a; /* Dark text color */
  -webkit-font-smoothing: antialiased; /* Improves font rendering on WebKit browsers */
}
```

FilterBar.module.css

Found in folder: **/src/components**

This CSS code styles a filter bar component with interactive buttons and a count display. It defines layout properties for the bar container and visual styles for filter buttons in default, hover, and active states. The count element is positioned to the right with subdued styling.

```
.bar {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 16px;
  flex-wrap: wrap;
}
.filterButton {
  padding: 6px 14px;
  font-size: 0.875rem;
  border: 2px solid #d1d5db;
```

```
  border-radius: 20px;
  background: #fff;
  color: #374151;
  cursor: pointer;
  transition: all 0.15s;
}
.filterButton:hover {
  border-color: #6366f1;
  color: #6366f1;
}
.filterButton.active {
  background: #6366f1;
  border-color: #6366f1;
  color: #fff;
}
.count {
  margin-left: auto; /* Pushes count to the right side of the flex container */
  font-size: 0.875rem;
  color: #9ca3af;
}
```

TaskInput.module.css

Found in folder: **/src/components**

This CSS file defines styles for a form component with an input field, a button, and a character counter. It styles the layout, focus states, and hover effects for these elements. It also includes commented-out, optional styles for a date input and a responsive layout.

```
.wrapper {
  margin-bottom: 16px;
```

```
}
.inputRow {
  display: flex;
  gap: 8px; /* Space between the input and button */
}
.input {
  flex: 1; /* Allows input to grow and fill available space */
  padding: 10px 14px;
  font-size: 1rem;
  border: 2px solid #d1d5db;
  border-radius: 6px;
  outline: none;
  background: #fff;
  transition: border-color 0.15s; /* Smooth color change on focus */
}
.input:focus {
  border-color: #6366f1; /* Highlight the input when focused */
}
.button {
  padding: 10px 18px;
  font-size: 1rem;
  font-weight: 600;
  background: #6366f1;
  color: #fff;
  border: none;
  border-radius: 6px;
```

```
  cursor: pointer;
  white-space: nowrap; /* Prevents button text from wrapping */
  transition: background 0.15s;
}
.button:hover {
  background: #4f46e5; /* Darker shade on hover */
}
.counter {
  margin: 4px 0 0;
  font-size: 0.8rem;
  color: #6b7280;
  text-align: right;
}
.atLimit {
  color: #ef4444; /* Red color to indicate a limit is reached */
  font-weight: 600;
}
/*
Optional due-date reference (inactive):
.input,
.dateInput {
  flex: 1;
  padding: 10px 14px;
  font-size: 1rem;
  font-family: inherit;
  border: 1px solid #d1d5db;
```

```
  border-radius: 6px;
  outline: none;
  transition: border-color 0.15s;
}
.dateInput {
  flex: 0 0 150px;
}
.input:focus,
.dateInput:focus {
  border-color: #6366f1;
}
@media (max-width: 520px) {
  .inputRow {
    flex-direction: column;
  }
  .dateInput {
    flex-basis: auto;
  }
}
*/
```

TaskItem.module.css

Found in folder: **/src/components**

This CSS file styles a list item component, likely for a to-do or task management interface. It defines visual styles for the item container, checkbox, text, edit input, and delete button, including hover effects and state changes. The commented-out section shows optional styling for due dates that is currently inactive.

```
.item {
```

```
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 14px;
  background: #fff;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  transition: background 0.1s;
}
.item:hover {
  background: #f9fafb; /* Light gray background on hover */
}
.checkbox {
  width: 18px;
  height: 18px;
  flex-shrink: 0;
  cursor: pointer;
  accent-color: #6366f1; /* Custom checkbox color */
}
.text {
  flex: 1;
  font-size: 0.95rem;
  color: #1a1a1a;
  word-break: break-word;
  cursor: text; /* Indicates text can be selected/edited */
}
```

```
.completed .text {
  text-decoration: line-through;
  color: #9ca3af; /* Grayed out text for completed items */
}
.editInput {
  flex: 1;
  min-width: 0;
  padding: 6px 8px;
  font-size: 0.95rem;
  color: #1a1a1a;
  background: #fff;
  border: 2px solid #6366f1; /* Focus border for editing */
  border-radius: 6px;
  outline: none;
  transition: border-color 0.15s;
}
.deleteButton {
  flex-shrink: 0;
  padding: 4px 8px;
  font-size: 0.75rem;
  background: transparent;
  border: 1px solid transparent;
  border-radius: 4px;
  cursor: pointer;
  color: #9ca3af;
  line-height: 1;
```

```
  transition: all 0.15s;
}
.deleteButton:hover {
  background: #fee2e2;
  border-color: #fca5a5;
  color: #ef4444; /* Red color on hover for delete action */
}
/*
Optional due-date reference (inactive):
.details {
  flex: 1;
  min-width: 0;
  display: flex;
  align-items: baseline;
  gap: 8px;
  flex-wrap: wrap;
}
.dueDate {
  flex-shrink: 0;
  font-size: 0.78rem;
  color: #6b7280;
}
*/
```

TaskList.module.css

Found in folder: **/src/components**

This CSS code defines two class selectors: `.list` styles an unordered list container with flexbox layout and spacing, while `.empty` styles a placeholder message for when the list has no items.

```
.list {
  list-style: none;     /* Remove default bullet points */
  margin: 0;            /* Remove default margin */
  padding: 0;           /* Remove default padding */
  display: flex;        /* Enable flexbox layout */
  flex-direction: column;/* Stack child elements vertically */
  gap: 8px;             /* Add spacing between child elements */
}
.empty {
  text-align: center;    /* Center-align text horizontally */
  color: #9ca3af;       /* Set muted gray text color */
  font-size: 0.9rem;     /* Use slightly smaller font size */
  padding: 40px 0;      /* Add vertical padding for spacing */
  margin: 0;            /* Remove default margin */
}
```

TaskNote.module.css

Found in folder: **/src/components**

This CSS defines two classes: `.note` creates a styled container with padding, a light background, and a bordered radius, while `.text` styles text within it to be smaller, gray, and to wrap correctly.

```
.note {
  margin: 0;
  padding: 8px 12px;
  background: #f9fafb;
```

```
  border: 1px solid #e5e7eb;
  border-radius: 6px;
}
.text {
  display: block;
  font-size: 0.85rem;
  color: #6b7280;
  word-break: break-word; /* Ensures long words or strings break to fit container */
}
```